

README

针对Pythia模型的高效推理实践

本项目在Pythia系列的模型上复现和评估大语言模型的高效推理技术，主要包含两个维度的优化：**精度控制（Precision Tuning）** 和 **投机采样（Speculative Decoding）**。通过对比实验，展示了不同配置下的推理速度、显存占用及模型质量变化，并在baseline的基础上有明显的性能提升。

项目结构

代码块

```
1  Pythia_accelerate/
2  |—— precision/           # 精度测试模块
3  |   |—— precision_eval.py # 精度评测脚本（FP32 vs FP16 vs BF16）
4  |   |—— ...              # 生成的图表
5  |—— spec_decode/        # 投机采样模块
6  |   |—— pg_19_eval.py    # PG-19 数据集评测脚本
7  |   |—— wikitext_eval.py # Wikitext-2 数据集评测脚本
8  |   |—— ...              # 生成的图表
```

快速开始

1. 环境依赖

请确保已安装 Python 3.10 及 CUDA 环境。安装所需依赖库：

代码块

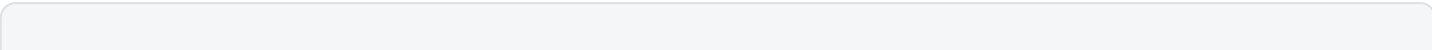
```
1  pip install torch transformers datasets matplotlib numpy tqdm seaborn
   accelerate
```

2. 运行代码

注意：所有脚本默认使用 `cuda:0`，在当前目录下的所有运行结果图片都是在单张3090显卡上运行的。

A. 精度基准测试 (Precision Benchmark)

对比 FP32、FP16、BF16 的推理速度与显存占用：



```
1 python LLM-Efficient-Reasoning/precision/precision_eval.py
```

- **输出：**终端显示各项指标，并生成 `precision_benchmark_2rows.png` 图表。

B. 投机采样测试 (Speculative Decoding)

使用小模型（Pythia-70M/160M）加速大模型（Pythia-2.8B）的推理。

运行 Wikitext-2 数据集测试：

代码块

```
1 python LLM-Efficient-Reasoning/spec_decode/wikitext_eval.py
```

运行 PG-19 数据集测试（长文本）：

代码块

```
1 python LLM-Efficient-Reasoning/spec_decode/pg_19_eval.py
```

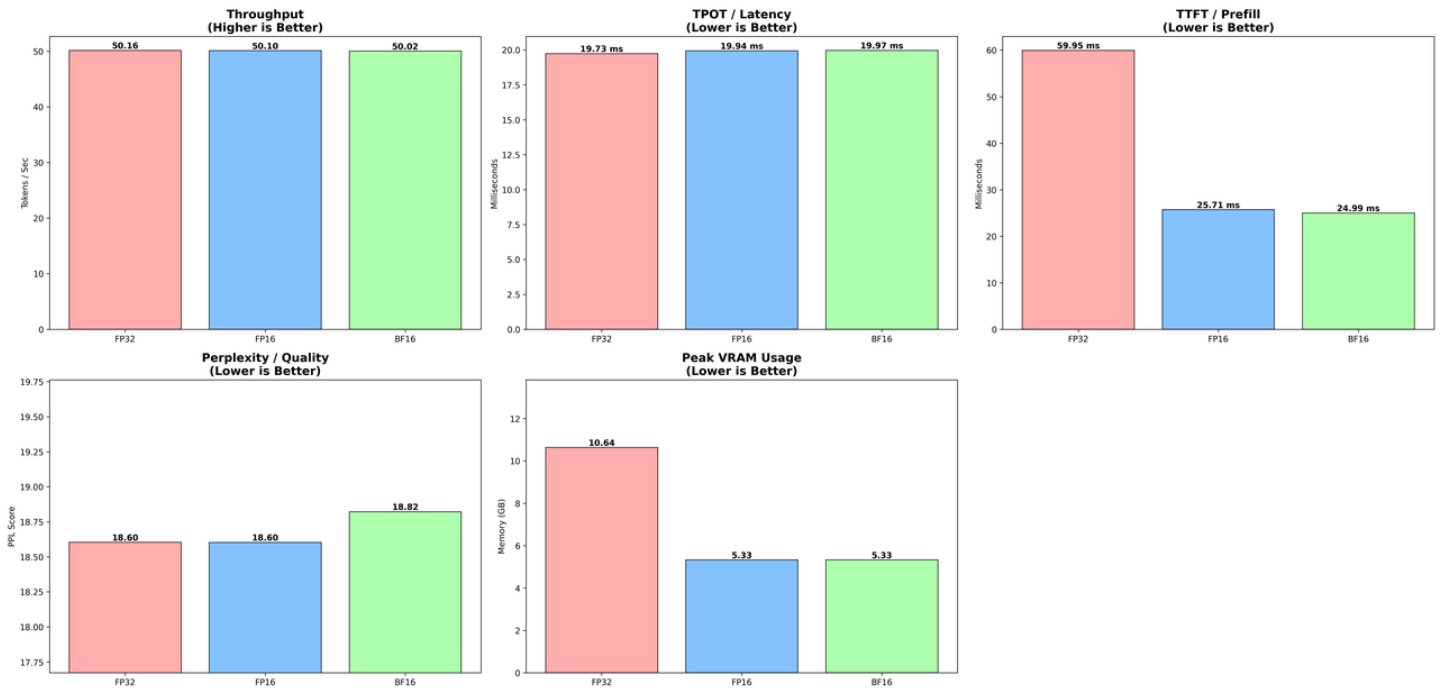
- **输出：**终端显示不同 K 值（猜测步数）下的加速比，并生成对比图表。

加速与优化效果

本项目通过以下实验展示了显著的推理加速效果：

1. 精度量化加速实验结果分析

基于 Pythia-2.8B 模型在 NVIDIA RTX 3090 上的实测数据，我对比了 FP32、FP16和 BF16三种格式的性能表现。实验结果揭示了精度切换对显存、延迟和吞吐量的不同影响机制：



- **显存占用**：大致下降为之前的一半，从 FP32 的 **10.64 GB** 降至 FP16/BF16 的 **5.33 GB**。
- **响应速度 (TTFT)**：首词延迟（TTFT）从 FP32 的 **64.3ms** 大幅下降至 FP16/BF16 的 **~26-28ms**，降幅超过 **58%**。
- **吞吐量 (Throughput)**：生成阶段的吞吐量在三种精度下均维持在 **~50 tokens/s** 左右，未见显著差异。
- **分析：为什么首字延迟有明显优化，吞吐量却没有多大影响：**
- Prefill 阶段是**计算密集型任务**，FP16/BF16 充分利用了 RTX 3090 的 Tensor Cores 加速能力，因此首词响应极快。
- Decoding 阶段是**访存密集型任务**。瓶颈主要在于显存带宽和 Python 循环开销，而非计算算力，因此半精度的计算优势未完全转化为吞吐量提升。
- **PPL**: FP32 和 FP16 的得分完全一致（均为 **18.60**），BF16 仅微幅上升至 **18.82**。

总结：对于 Pythia-2.8B 模型的实时推理场景，**FP16/BF16 是更优选择**。它在保证模型“智力”不下降的前提下，将显存成本减半，并将用户最敏感的首字响应速度提升了 2.3 倍。

2. 投机采样加速实验结果分析

基于 `wikitext_eval.py` 和 `pg_19_eval.py` 的多组对比实验，我深入验证并分析了 speculate decode 在不同场景下的效能。实验采用 **Pythia-70M/160M** 作为 Draft Model，**Pythia-2.8B** 作为 Target Model。

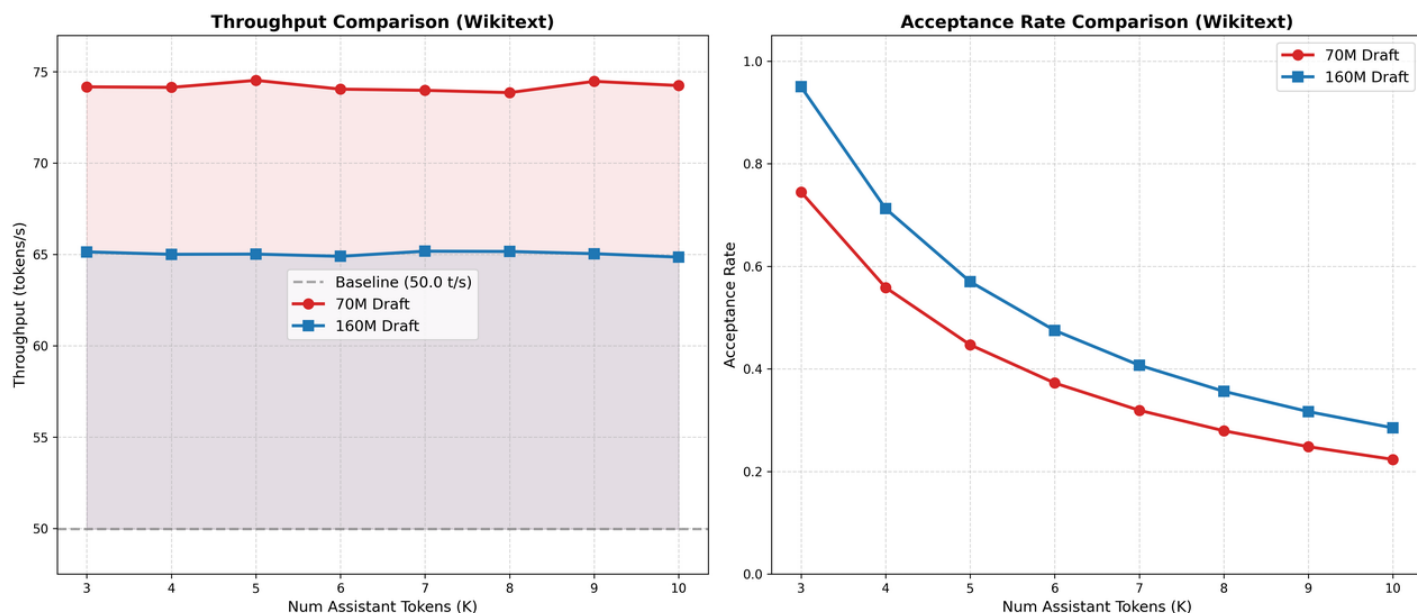


图 1: Wikitext 数据集上的投机采样性能对比

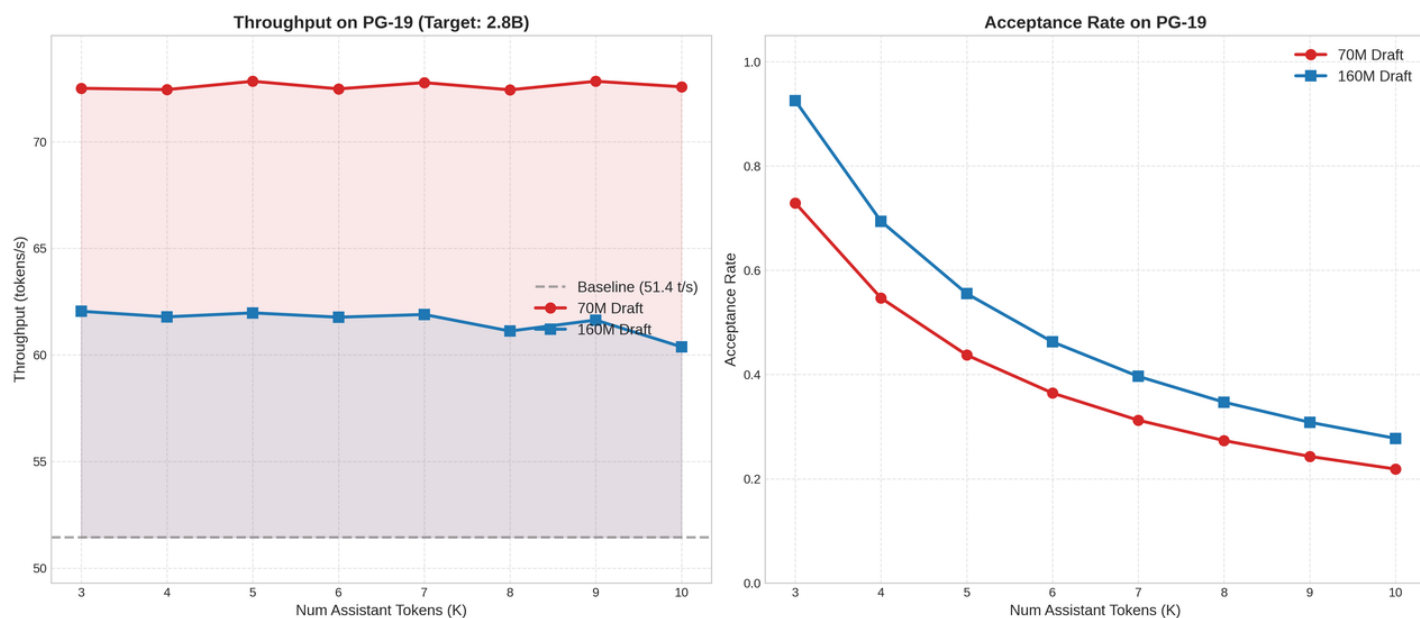


图 2: PG-19 数据集上的投机采样性能对比

- **核心原理与验证：** 实验成功复现了投机采样的核心优势：利用小模型的**低延迟**特性快速串行生成K个草稿 Token，再由大模型进行**并行验证**。
- **实验结果分析：**
- **数据集敏感性：** 实验表明加速比高度依赖于文本的“可预测性”，也就是对于大语言模型的“难度”。在简单数据集（Wikitext）上，Acceptance Rate较高，实现了约 **1.5x** 的端到端加速；而在复杂长难文本（PG-19）上，接受率显著下降，加速比回落至 **1.2x - 1.3x** 区间。

- **最佳步长 (Optimal K)**: 通过扫描，我并没有观察到吞吐量呈现典型的*“倒U型”曲线*。我想这是因为小模型本身也只能正确推理后面两三个词，所以后面的其实都没用，但是由于小模型耗时非常少，所以下降的不明显。
 - **模型选择的权衡**:
 - **Pythia-160M** (更聪明): 在 PG-19 等难数据集上拥有更高的 **Acceptance Rate**，证明其预测分布更接近大模型。
 - **Pythia-70M** (更轻量): 尽管接受率较低，但其自身的**推理延迟极低**。实验数据显示，在 2.8B 这一量级的目标模型下，**70M 草稿模型的Throughput反而高于 160M**。
 - **结论**: 在投机采样中，草稿模型的“生成速度”往往比“预测准确率”具有更高的权重。对于中小规模的目标模型，**极致轻量化的草稿模型**是更优选择。
-

实验配置

- **Target Model**: EleutherAI/pythia-2.8b
- **Draft Models**: EleutherAI/pythia-70m, EleutherAI/pythia-160m
- **Hardware**: Default `cuda:0` (Require ~8GB+ VRAM for benchmarks)，其中量化精度的效果在 3090或4090上更明显